RESEARCH CONTINUITY REPORT

Batched Oblivious Lookup in Ten-Party MPC

Square-root and tree ORAM, batched array access, honest-majority preprocessing, equality, and deployment costs

Consolidated from the research discussion
13 October 2026 (UTC)

Abstract

This report preserves the problem statement, candidate constructions, literature, derivations, corrections, and unresolved questions from a discussion of a database containing 2^{28} pairs of 32-byte keys and 32-byte values. The target is 10,000 read requests per second across ten MPC parties, using batches of approximately 1,024 requests and parallel workers. The security assumption evolved from conventional SPDZ-style dishonest-majority computation to allowing an honest majority. Degree-4 Shamir sharing among ten parties is therefore the main arithmetic representation considered here.

The central research direction is to combine efficient batched private array access with an enlarged square-root ORAM shelter and a shallow recursive position map. The report derives when this could outperform a direct batched scan, explains why compaction alone is insufficient, and records tree-based alternatives. It also gives explicit equality protocols and a conditional AWS cost model. No complete ten-party ORAM implementation or end-to-end throughput measurement was produced in the discussion. Numerical projections must retain their stated assumptions.

Reading guide. Start with Section **??**. The algorithmic models are in Sections **??**–**??**; equality is in Section **??**; cost accounting is in Section **??**. Section **??** identifies the next experiments.

Contents

1 Restart summary and status of conclusions

Numerical update, 13 October 2026. The user has now confirmed malicious security with abort and a five-second average-latency target, and requested both 2^{28} and 2^{32} records. The subsequent construction and capacity study is in `mpc_cost/mpc_oram_cost_report.tex`, with the rendered report at `output/pdf/mpc_oram_cost_report.pdf`. It supersedes the earlier numerical budget sketches below. All figures remain conditional models.

Annual USD, ten parties at 10,000 queries/s	2^{28}	2^{32}
Interactive Circuit-OPRAM reference	\$2.33B	\$5.16B
Modeled mean latency	4.77 s	3.97 s
Garbled RAM, selected prepared horizon	\$10.33–18.52B	\$21.59–38.71B
Garbled RAM, reference $T = 2N$ horizon	\$10.33–18.52B	\$55.07–65.69B

The interactive bottleneck is local arithmetic under the baseline phase attribution, with RAM capacity also constraining the larger case. In garbled RAM, about 97.7% of modeled communication is preprocessing; SSD retention can dominate the dollar bill for long prepared epochs. The selected horizons are 14.91 hours at both sizes, minimizing the conservative fleet endpoint within the tested grid. They are preparation horizons, not query-batching delays. Garbled estimates use zero stack/adaptor cost under the requested migration assumption, SSD storage, and networking as the throughput bottleneck. They exclude a concrete finite correlation generator and complete malicious initialization/epoch renewal; the range reflects party traffic imbalance only. The full study includes exact-key and indexed parallel-scan comparisons, Python calculations, sources, and sensitivity analyses. The remainder of this document preserves the earlier research discussion rather than asserting that its alternatives were implemented.

Working direction. Prototype an honest-majority arithmetic backend with batched gather/merge primitives before selecting the ORAM. Compare a direct batched array access, a recursive square-root ORAM with an enlarged shelter, and a tree ORAM with parallelized metadata processing using the same field, security threshold, payload encoding, and complete preprocessing costs.

The following points should survive into later sessions.

1. **The user accepts honest majority.** Ten parties with degree-4 Shamir sharing suggests privacy against up to four corruptions. Malicious security with abort is now the user-confirmed requirement. Do not silently substitute a three-corruption protocol requiring $t < n/3$, or assume passive security is sufficient.
2. **The square-root proposal is promising but unbenchmarked.** An idealized shelter/rebuild model gives $S \asymp \sqrt{Db}$, where D is dense array capacity and b is the number of dense accesses in a batch. For one dense access per request, $D = 2^{28}$ and $b = 2^{10}$ give $S = 2^{19}$. This is a derived balance, not a measured optimum.
3. **The fair baseline is a batched gather.** It need not perform Db equalities. Private stable sorting/routing, finite differences, and local prefix sums give a substantially better baseline. Compaction requires selection marks; it does not create membership predicates for free.
4. **Duplicate handling depends on the construction.** A direct batched read can support duplicates without an explicit deduplication pass. An ORAM batch must still avoid exposing repeated physical probes or hit counts. Recursive position-map accesses can duplicate even when data addresses differ.
5. **Equality need not decompose both inputs.** A fresh random field mask with shared canonical bits yields an exact secret equality bit. A simple product-tree implementation takes $\ell - 1$ secure multiplications after one opening for an ℓ-bit prime field. A separately derived power-tuple variant moves this product work offline and uses two dependent online openings.

6. **Honest majority changes the preprocessing economics.** Conventional dishonest-majority LowGear/SPDZ triple rates are not the correct baseline. ATLAS and Shamir-based malicious protocols are more relevant, but published synthetic-circuit rates are not ORAM throughput measurements.
7. **No final annual budget has been established.** The earlier rough $0.3–1 billion/year dishonest-majority projection is superseded as a baseline. The honest-majority model remains conditional on the as-yet-unmeasured work per lookup and deployment topology. Cross-AZ bandwidth can itself dominate.

1.1 Evidence labels

Label	Meaning in this report
Requirement	Explicitly supplied by the user.
Working assumption	Chosen to make an analysis concrete; still subject to design decisions.
Published result	Attributed to an identified paper or maintained implementation.
Derived model	Algebra or a construction assembled in the discussion, with assumptions stated.
Unresolved	Requires a proof, implementation, parameter choice, or measurement.

The report is a technical continuity record, not a security proof for a complete system. Absence of a located paper with the exact proposed combination is not evidence of novelty. Literature coverage reflects the discussion and selected checks through the consolidation date.

2 Workload, notation, and security model

Symbol	Value or role	Interpretation
N	2^{28}	Number of logical key–value pairs.
Key/value	$32 + 32$ bytes	Sparse 256-bit keys, 256-bit values.
Q	$1024 = 2^{10}$	Logical map requests per batch.
R	10,000 s^{-1}	Target aggregate logical read rate.
n, t	$10, 4$	Parties and working corruption threshold.
D	Construction-dependent	Number of dense cells after map representation.
h	Usually 1 or 2	Dense accesses per logical map request.
b	hQ	Dense accesses per batch.
S	Tunable	Shelter capacity / padded accesses per epoch.
F_j	Tunable	Number of position labels packed into a recursive block.
w	Context-dependent	Field-word width in bytes, or payload width when explicitly stated.
G	Unmeasured	Full-backend field multiplications per logical lookup.

The raw database size is

$$N(32 + 32) = 2^{34} \text{ bytes} = 16 \text{ GiB}.$$

This excludes hash-table slack, ORAM buckets, dummy cells, position maps, authentication, field-encoding overhead, shelter storage, and worker copies. For example, a dense table with $D \simeq 2N$ and

64-byte cells already occupies 32 GiB before those additions. Field shares are not generally packed as the original bytes: encoding a 256-bit value in five 61-bit field elements consumes 40 bytes per party before any tags.

2.1 Batch semantics and latency

The main analysis assumes *nonadaptive batches*: all addresses in a batch are available before its evaluation. Dependent traversals within one lookup still require sequential phases. Batching independent lookups fills circuit layers and amortizes overhead; it does not remove a data dependency.

At 10,000 requests/s, a single stream takes $Q/R = 102.4$ ms to accumulate 1,024 requests. Uniform arrivals give roughly 51.2 ms average batch-formation waiting time before computation. Splitting traffic across many independent workers increases this delay unless batches are formed centrally, smaller batches are used, or sufficient additional concurrency is available. This matters when combining a throughput target with a future latency objective.

Application writes are out of scope for the cost target. *ORAM state still changes on reads*: locations are remapped, shelters updated, and arrays shuffled or rebuilt. Immutable logical data makes replication easier but does not make an individual ORAM replica stateless.

2.2 What honest majority does and does not fix

Degree-4 Shamir sharing among ten parties permits the usual honest-majority multiplication regime because $2t = 8 < n = 10$. This is a statement about sharing and the availability of suitable protocols. Active correctness requires verified preprocessing, consistency checks, and an appropriate opening procedure.

The desired leakage should include public database capacity, batch size, and a fixed service schedule, but exclude logical addresses, keys, equality outcomes, hit/miss status, duplicate structure, and the number of distinct requests unless explicitly allowed. Aborts and output release must follow the chosen malicious protocol. Using one AWS provider or one availability zone is an operational assumption; it is not a proof of independence between the parties.

3 A common accounting framework

For each candidate, report at least the vector

$$(M_{\text{field}}, M_{\text{bit}}, C_{\text{eq}}, C_{\text{order}}, W_{\text{perm}}, B_{\text{comm}}, D_{\text{interactive}}, M_{\text{RAM}}),$$

where the entries denote arithmetic work, Boolean work, equality and ordering operations, words privately permuted, bytes communicated, interactive depth, and memory. A scalar cost is useful only after assigning backend-specific weights. In particular, ordering a secret address and moving one payload word are different operations.

Conditional movement of one field word has the form

$$\text{mux}(s, x, y) = y + s(x - y),$$

so a selector applied to a multiword record generally entails multiple secret products. Reusing the selector may enable vectorized authentication or checks, but the record width still matters. Conversely,

moving local share arrays by a public index costs no MPC multiplication. The cost of a *private* permutation depends on how its control information is represented and prepared.

Maintain four separate ledgers:

1. online work and dependent communication phases;
2. preprocessing production, verification, and storage;
3. amortized rebuild and position-map maintenance;
4. steady-state wall time, RAM traffic, network traffic, and cloud charges.

An online-only speedup is insufficient for a continuously operating service if preprocessing cannot be replenished at the same rate. An annual estimate must also account for the cost of all ten parties rather than just one.

4 Direct batched access, compaction, and duplicate requests

4.1 The direct baseline is not a batch of independent scans

Performing b independent scans of D records costs $\Theta(Db)$ predicates. That is a useful simple implementation, but an unfair sole baseline for the proposed batched ORAM. Laud's arithmetic-black-box array access gives a stronger alternative using private stable sorting and permutation operations [**?**]. Its read construction can be expressed as follows.

Prepare a stable sorting permutation π for

$$(0, 1, \ldots, D-1, a_0, \ldots, a_{b-1}),$$

where database indices are public and query indices are secret. Form

$$\Delta X = (X_0, X_1 - X_0, \ldots, X_{D-1} - X_{D-2}),$$

append b zero records, apply π, take prefix sums locally on the shares, and apply π^{-1}. The query positions contain their requested values because

$$\sum_{i \le a} \Delta X_i = X_a.$$

Stability puts a database row before its equal-address queries. Repeated queries add zeros, so all obtain the same answer. This read method does not require a separate query deduplication step or secret conditional copying of every payload word in a segmented scan.

For accounting, write

$$F(D, b, w) = \text{PrepareRead}(D, b) + 2\,\text{ApplyPerm}(D + b, w) + O(Dw)_{\text{local}}. \tag{1}$$

The preparation term is essential. A generic oblivious sorting network may dominate it. More specialized stable merge/radix/shuffle constructions may be better, depending on the MPC backend. Computing a narrow permutation before applying it to wide records avoids charging the entire payload width at every metadata-sorting stage. The map from 256-bit keys to dense indices is a separate problem; this baseline is immediately applicable to an indexed array and to recursive position maps.

4.2 What compaction actually contributes

Oblivious compaction accepts secret marks and moves marked records to designated positions without revealing the marks. It does not decide which records match the secret requests. If producing each mark still compares a record with every query, the Db work remains. A meaningful claimed improvement must provide the complete private join, routing, or membership procedure.

Stable compaction, merging, and secure shuffles are therefore relevant building blocks [**?**, **?**, **?**, **?**], but they have different party counts, security assumptions, online/offline splits, and output formats. A two-party or passive three-party timing cannot be inserted into the ten-party malicious model without an implementation and security adaptation.

Stable three-party sorting [**?**] and sorting-based private JOIN/GROUP-BY [**?**] provide relevant database precedents for separating narrow control from wide records. Linear-work tight compaction is also possible in an oblivious parallel model [**?**]; neither an inherent $N \log N$ compaction cost nor free matching should be assumed. Stability, cardinality leakage, and output multiplicity depend on the chosen primitive.

4.3 A sorted-shelter extension

Status: Derived construction idea. Suppose the shelter contains valid records sorted by secret logical address. Stable-merge its addresses with the batch addresses, placing a shelter record before equal queries. Apply the same finite-difference and prefix-sum idea to the shelter's address, value, and validity columns. After inverse routing, each query receives its predecessor's columns. Its membership bit is

$$\mathrm{hit}_j = \mathrm{predValid}_j \cdot \mathrm{EQ}(\mathrm{predAddr}_j, a_j).$$

This needs only b final equalities, rather than Sb equality tests. A validity-0 sentinel below the valid address domain handles queries with no predecessor; invalid padding must sort after valid addresses. Sentinel and padding order should use an explicit tagged representation rather than an accidental field ordering convention.

The unresolved cost is the private stable merge permutation, its application and inverse, and maintenance of the sorted shelter. A comparator-based merge can still contain many expensive secret ordering operations. Thus the extension does not establish linear cost with a small constant on the target backend.

4.4 Deduplication and inverse routing

An ORAM needs more than a functional batched read: the publicly observed physical accesses must not expose repeats or shelter hits. A typical batch wrapper sorts the query addresses, forms secret group-boundary bits, chooses representatives, performs the real operations with fixed-size cover traffic, copies answers to duplicates, and restores original order. Counts and group structure remain secret. All of these steps belong in the cost.

For scale, a bitonic sorter on $b = 1024$ inputs has

$$\frac{b}{4} \log_2 b(\log_2 b + 1) = 28{,}160$$

compare-exchanges and 55 comparator layers. This is a metadata reference count, not a recommendation to use bitonic sorting, and not an arithmetic multiplication count. For $b = 2048$, the corresponding counts are 67,584 and 66.

With read-only data and a unique shelter, inserting only distinct real misses avoids a full-shelter adjacent-equality pass: these addresses are absent from the old shelter and mutually distinct. There is a qualification if cover probes fetch unused *real* rows instead of explicit dummy blocks. Such rows must also be cached in the shelter, or made available through another equally oblivious mechanism, so future requests need not revisit their exposed physical locations. Fresh real cover probes must also avoid the batch's other fresh probes, including true misses. Cover probes, duplicate suppression, and dummy capacity need one coherent security argument.

5 Square-root ORAM: balance and comparison with direct access

5.1 Construction skeleton and known starting point

The proposal uses a shuffled main array, a shelter of touched records, and a fixed rebuild schedule. A query is resolved privately against the shelter; otherwise it accesses the appropriate as-yet-unexposed main-array location. Cover accesses hide which case occurred. The position map is itself recursively protected, with a flat batched-access base case. Zahur et al.'s *Revisiting Square-Root ORAM* is a concrete MPC-oriented starting point with recursive permutation maps and bulk initialization [**?**].

The particular combination proposed here—enlarged shelter, batched private shelter merge, high position-map fanout, and a large base case on an honest-majority backend—has not been implemented or fully proved in this discussion. The following analysis is conditional on realizing its component costs and access schedule securely.

5.2 General amortization model

Let $J(S, b)$ be the *complete* batch shelter operation, including predicates, routing, duplicate handling, output propagation, and maintenance as appropriate. Let $R_{\text{build}}(D)$ be a complete main-array rebuild including its position-map work. After a public schedule of approximately S/b batches, rebuilding yields

$$C_{\text{sqrt,batch}}(S) = J(S, b) + \frac{b}{S} R_{\text{build}}(D), \tag{2}$$

$$C_{\text{sqrt,map}}(S) = \frac{J(S, b)}{Q} + \frac{h}{S} R_{\text{build}}(D). \tag{3}$$

Extra fixed per-access work and recursion must be added. The epoch must be scheduled by public capacity, not by revealing the number of distinct misses. These equations use one consistent scalar resource; they cannot mix online latency in J with total communication in R_{build}.

Suppose the measured range admits

$$J(S, b) \simeq c_H S + H(b), \qquad R_{\text{build}}(D) \simeq c_R D.$$

Then differentiating gives

$$S^* = \sqrt{\frac{c_R}{c_H} D b}, \tag{4}$$

$$C^*_{\text{sqrt,batch}} = 2\sqrt{c_H c_R D b} + H(b). \tag{5}$$

Thus $S \asymp \sqrt{Db}$ is the equal-coefficient specialization. If rebuilding costs $\Theta(D \log D)$ while shelter work is linear, the balance instead becomes $\Theta(\sqrt{Db \log D})$. With logarithmic shelter routing, optimize the

actual expression; in general the stationary condition is

$$\frac{\partial J(S,b)}{\partial S} = \frac{bR_{\text{build}}(D)}{S^2}.$$

For whole-batch epochs, require $S \geq b$ and a public batch-aligned schedule; round or project the stationary optimum into the construction's feasible range. Include material dummy padding in the actual rebuilt capacity. If the stationary point is infeasible, its formula is only an unconstrained lower value for the model, and the constrained cost must be evaluated instead.

Illustrative layout	D	b	$S = \sqrt{Db}$	Raw shelter
One dense access	2^{28}	2^{10}	2^{19}	32 MiB
Two-candidate table	2^{29}	2^{11}	2^{20}	64 MiB

Table 1: Equal-coefficient examples counting 64-byte raw shelter payloads only; addresses and validity metadata are additional. The second row assumes $D \simeq 2N$ only for sizing; actual hashing parameters must be chosen.

Both rows have 512 batches per epoch, or approximately 52.43 seconds for one replica receiving all 10,000 logical requests/s. With K equally loaded independent replicas, each epoch lasts approximately $52.43K$ seconds; aggregate amortized rebuild work is unchanged, while state replication increases. Main-array dummy capacity and payload/share encoding may increase these raw sizes. Deamortizing rebuilding introduces additional memory and scheduling costs; the table gives no tail-latency guarantee.

5.3 Crossover against an efficient direct batch

For a local approximation $F(D,b) \simeq c_F D$, set

$$\alpha = c_H/c_F, \qquad \beta = c_R/c_F.$$

Ignoring additive and recursive differences, the optimally tuned ratio is

$$\frac{C^*_{\text{sqrt,batch}}}{F(D,b)} = 2\sqrt{\alpha\beta\, b/D}. \tag{6}$$

For both tabled layouts $D/b = 2^{18}$, hence

$$\frac{C^*_{\text{sqrt,batch}}}{F(D,b)} = \frac{\sqrt{\alpha\beta}}{256}.$$

When the stationary optimum is feasible, the simplified model favors square-root ORAM when $\alpha\beta < 65{,}536$. At the untuned choice $S = \sqrt{Db}$ the ratio is instead $(\alpha+\beta)/512$. With equal constants the modeled ratio is $1/256$; *this is not an end-to-end speedup prediction.*

The practical question is whether the private shelter join, rebuild, recursion, and online physical accesses fit within this headroom when compared with a well-implemented direct gather. A direct scan may have excellent memory locality and simpler preprocessing. The model establishes a plausible research direction, not a winner. Benchmarking only a naive Db scan would not answer the user's question.

6 Recursive position maps and the large-base proposal

Let ORAM level 0 contain $n_0 = D$ records. Pack F_j position labels into a record at the next level, so

$$n_{j+1} = \lceil n_j / F_j \rceil .$$

If each label uses one field element, a level-$(j+1)$ record carries approximately F_j such elements. Fewer records therefore do not imply proportionally fewer payload words. A packed block also requires private subblock selection. In the lookup-only square-root construction, stored parent position labels remain fixed within the parent epoch and are replaced during bulk parent reconstruction. The recursive map's own shelter/access metadata still changes. Tree ORAM may instead update a retrieved label on each access.

If levels $0, \ldots, L$ are ORAMs and the position map of level L is flat, the flat map has n_L scalar labels. Stopping immediately at level 0 leaves a D-label map. One cannot count a D/F_0-entry base without introducing and paying for the additional packed ORAM level.

A useful batch accounting template is

$$\begin{aligned} C_{\text{recursive}} = & \sum_{j=0}^{L} \left(J_j(S_j, b) + \frac{b}{S_j} R_j(n_j, w_j) \right) + F_{\text{base}}(n_L, b) \\ & + \sum_{j=0}^{L-1} \text{SubblockSelect}_j(b, F_j) + \text{GroupingAndRouting} . \end{aligned} \tag{7}$$

Here using b at every level is a padded schedule; a more refined construction may exploit private grouping but must not reveal the number of distinct blocks. Parent rebuilds can force descendant reconstruction. Those costs must be counted once, at their actual frequency, rather than assigning independent optimal periods to all levels. In the one-level model, R_{build} may aggregate all work triggered by that rebuild; in the recursive sum, do not count that same descendant work again. Likewise, let J_j be level-local and assign cross-level grouping/routing to one term. Bulk rebuilds are essential; issuing a separate recursive write per parent entry can destroy the intended savings.

6.1 Why larger fanout can help, and where it stops helping

Status: Derived coefficient model. For n labels packed F per record, suppose shelter work per record is $\gamma + \mu F$ and rebuild work per record is $\rho + \nu F$. Optimizing that added level gives

$$2\sqrt{bn\left(\frac{\gamma\rho}{F} + \gamma\nu + \mu\rho + \mu\nu F\right)}. \tag{8}$$

If comparison/control dominates shelter work and payload dominates rebuilding, the cross term can dominate and the cost is approximately insensitive to F. Larger fanout may then reduce recursive depth and base size cheaply. Once both phases are payload-dominated, the $\mu\nu F$ term penalizes excessive fanout. Subblock extraction can impose an additional penalty absent from this simplified formula.

Choose the flat cutoff by comparing its *batched* gather with the complete cost of one more level plus its smaller base. A large base is plausible because batched gather is much cheaper than b independent scans. It is not justified by a universal asymptotic cutoff. Also, a shared epoch size eventually exceeds the capacity of a small recursive level: stop recursion, explicitly pad that level, or design a valid nested schedule.

7 Tree ORAM, eviction, and sparse-key maps

7.1 Circuit ORAM and the depth of eviction

Circuit ORAM is a natural small-circuit reference [**?**]. The concern is that its eviction planning contains dependent scans and metadata comparisons that are relatively expensive in arithmetic MPC. Nevertheless, sequential-looking pseudocode does not imply an inherent $\Theta(\log D)$ interactive depth for every eviction implementation.

Chan, Chung, and Shi explicitly parallelize Circuit ORAM's planning and eviction in the full version of *On the Depth of Oblivious Parallel RAM*, Section 7.1 [**?**]. For a path with L levels they obtain $O(\log L)$ word depth and $O(L \log L)$ work while preserving the sequential eviction outcome. This is the most directly relevant literature answer to the eviction-depth question. It does not mean $\lceil \log_2 28 \rceil = 5$ SPDZ rounds: a word comparison, lookup, or conditional move must itself be compiled, and paths interact through buckets and stash state.

Titanium's optimized Circuit ORAM eviction reduces payload conditional operations using combined pick/drop swaps [**?**]. This addresses a different part of the cost from parallelizing the planning scan. A useful implementation should measure metadata planning and wide-record movement separately, then investigate whether their optimizations compose.

7.2 Arithmetic-friendly reformulations to test

Status: Implementation ideas, not measured improvements. For a public eviction path, compute each block's eligibility for all relevant depths using the bits of its secret leaf label and public path bits. Shared prefix-match bits can be formed by a parallel prefix product rather than repeatedly comparing full integers. Keep already-needed metadata in bit form where this avoids repeated arithmetic-to-bit conversion.

Maxima, first/last eligible positions, and selected-candidate propagation can often be written as associative scans on tagged summaries. The combination operator must be associative with explicit tie-breaking and validity rules; merely calling a loop a prefix scan does not establish this. The published parallel Circuit ORAM analysis is the correct starting point. A resulting design still needs a stash-overflow argument and proof that its eviction schedule preserves the ORAM invariant.

Appendix **??** retains more explicit proposed scan formulations from the discussion. Eviction's bucket-depth/target comparisons involve roughly five-bit control words at the target scale, rather than 256-bit map keys. Leaf compatibility uses the longer leaf labels. Storing every leaf bit as a separate arithmetic share can substantially expand metadata; compact storage and free bit access cannot both be assumed.

Efficient Mixed-Mode Oblivious RAMs also proposes parallel bucket-merge eviction [**?**]. It trades dependence depth against larger buckets and merge circuits. Its specialized three-party design is a candidate to study, not an established malicious ten-party replacement.

Ring ORAM is worth comparing because it shifts work into bucket reshuffling and scheduled eviction [**?**]. However, its client/server bandwidth savings do not directly imply a smaller malicious MPC circuit. Private slot selection, validity bookkeeping, reshuffling, and authentication may dominate in this setting. No verified alternative was found that simultaneously guarantees Circuit-ORAM-like concrete cost and a superior ten-party arithmetic-MPC implementation for this workload.

7.3 What a batch of paths can save

If b independently uniform leaf paths in a binary tree are fetched together, the expected number of distinct nodes at depth d is

$$2^d \left(1 - (1 - 2^{-d})^b\right). \tag{9}$$

Summing over depths gives approximately $b(\log_2(D/b) + O(1))$ nodes rather than $b \log_2 D$, up to the relation between leaf count and array capacity. This is an occupancy calculation, not a complete batched ORAM algorithm. Shared upper nodes offer limited asymptotic savings at $D = 2^{28}, b = 2^{10}$: roughly the lower 18 levels remain distinct per query. Repeated logical requests, adaptive position-map lookups, and conflicting evictions need additional coordination. Public paths must have the distribution required by the ORAM security proof. For height 28 and 1,024 independent paths, Equation (**??**) gives 19,568.31 expected distinct buckets versus 29,696 separate visits, a 34.10% reduction in this count alone. Eviction paths have their own schedule and must not automatically be assigned the same random-path model.

Jarecki and Wei study three-party Circuit ORAM with fast batch retrieval [**?**]; FABLE studies efficient batched private lookup in a two-party setting [**?**]. Both are relevant design references, but their concrete protocol costs do not transfer directly to ten malicious parties. Jarecki–Wei's stated fast-batch regime is $b = O(\log D)$ and tolerates one semi-honest corruption among three parties; using batch 1,024 requires examining its schedule and stash constraints. FABLE is semi-honest two-party computation with a table held by one party, and its server still performs linear table work. Its low communication should not be interpreted as sublinear total computation.

7.4 Sparse 256-bit keys: cuckoo table versus an oblivious search tree

Cuckoo representation. Compute two candidate indices under suitable hash functions/PRFs inside MPC and read both table locations obliviously. Compare the stored full keys, then select the value or a missing-key result. Charge two dense accesses, hash-table slack, PRF evaluation, collision handling, any hashing stash, and exact key equality. "Two ORAM accesses" describes the ordinary two-candidate lookup only; it omits any overflow structure. Initialization and failure parameters must be selected for 2^{28} entries. The original two-choice cuckoo construction is a useful reference [**?**]; one-slot variants require operating slack around twice the logical capacity, while bucketed variants change the number of candidate records compared. Both probes can be evaluated in parallel.

The phrase "MPC-friendly PRF" is a design requirement, not a selected primitive. The relevant metric is arithmetic cost and depth over the actual field, with appropriate security parameters and domain separation for the two candidates. Public deterministic hashing may be possible under a different access/leakage argument; it must not be substituted without specifying that argument. A static dictionary or perfect-hash construction is also a possible future read-only alternative, but was not analyzed in this discussion. MiMC and Poseidon are candidate field-native primitive families [**?**, **?**], not a selected keyed instantiation. Encoding 256-bit inputs, keyed/domain-separated use, range reduction to array indices, and multiplication depth all need to be specified and costed.

Search tree integrated with location information. EnigMap is relevant to overlapping search-tree navigation and ORAM position information [**?**]. This can avoid treating every tree edge as an independent access to a fully recursive generic ORAM. Its secure-enclave/client setting differs from arithmetic MPC. Moreover, its paper's batch requests are processed sequentially; a large-batch benchmark does not establish the batched MPC amortization needed here. The underlying pointer/position-map overlap originates in oblivious data structures [**?**]. A literal EnigMap node contains key-bearing pointers for itself and two children; with 32-byte keys and a 32-byte value this can already mean 128 bytes before

position labels. This is not a lower bound: dense node identifiers may permit a different encoding. Do not assume the search-tree node has the same width as the original 64-byte pair.

For this workload a binary search tree entails many dependent comparisons on long keys unless the comparison representation is redesigned. The choice is therefore between a small number of hashed dense lookups plus exact equality and dependent search/navigation with potentially fewer redundant position-map operations. Measure both the key-processing cost and the memory-access cost; do not compare only the number of ORAM calls.

8 Honest-majority arithmetic and degree-4 sharing

8.1 Multiplication is not local share multiplication alone

Let party i hold $f_x(\alpha_i)$ and $f_y(\alpha_i)$, with degree-4 polynomials and constants x, y. Their local product is a point on $f_x f_y$, which has degree at most eight. Returning that value as a fresh degree-4 share is incorrect. Repeated local multiplication rapidly exceeds the reconstruction threshold.

One standard route uses correlated sharings of the same fresh random r at degrees four and eight. Open a properly randomized sharing of $xy + r$, then subtract $[r]_4$ to obtain a degree-4 result. Another route uses a checked Beaver triple $([a]_4, [b]_4, [ab]_4)$: open $\delta = x - a$ and $\epsilon = y - b$, then compute

$$[xy]_4 = [ab]_4 + \delta\,[b]_4 + \epsilon\,[a]_4 + \delta\epsilon. \tag{10}$$

All output terms remain degree four. The opening and preprocessing subprotocols must match the active-security model; naive interpolation is not a way to correct four malicious errors in a degree-8 sharing among ten parties. In particular, opening the raw degree-8 product polynomial can reveal information beyond its constant term. Use the backend multiplication primitive.

8.2 Practical starting points

MP-SPDZ supports several distinct backends; its name does not imply that every backend uses conventional dishonest-majority SPDZ [**?**].

Executable	Role in the proposed benchmark
`malicious-shamir-party.x`	Malicious prime-field Shamir baseline.
`sy-shamir-party.x`	Malicious SPDZ-wise Shamir baseline with authentication.
`shamir-party.x`	Passive baseline; not a substitute for active security.
`atlas-party.x`	Documented MP-SPDZ implementation is passive, despite the ATLAS paper also giving malicious protocols.

The Shamir options permit $n = 10, t = 4$; pin a source revision and explicitly set party count and threshold when benchmarking [**?**]. Foundational malicious honest-majority references include Lindell–Nof and Chida et al. [**?**, **?**]. No benchmark in the discussion establishes which current malicious backend wins on this database circuit.

8.3 Batching is not automatically packed Shamir sharing

With ordinary polynomial packing, degree d supports k secret slots and privacy threshold $t = d - k + 1$. Local products require $2d < n$ for ordinary degree reduction. Hence

$$k \le \left\lfloor \frac{n-1}{2} \right\rfloor - t + 1.$$

At $n = 10, t = 4$, this gives $k \le 1$. Thus a batch of 1,024 requests cannot simply be packed into a single ordinary degree-4 Shamir polynomial while retaining four-corruption privacy. Advanced sharing transformations, different sharing schemes, or weaker thresholds can change the situation. This elementary bound does not rule out the advanced packed protocols surveyed below.

9 Equality on degree-4 Shamir sharings

The output required by an ORAM is usually a *secret* bit $[e]_4$, where $e = 1$ iff $x = y$. Revealing this bit can expose hit/miss or duplicate information. All secure products below return degree-4 sharings through a valid backend, and all fresh correlations require the backend's active checks.

9.1 Exact full-field masked equality

Let $x, y \in \mathbb{F}_p$ and $\ell = \lceil \log_2 p \rceil$. Prepare a fresh uniform $r \in \mathbb{F}_p$ together with degree-4 shares of its *canonical* bits:

$$r = \sum_{j=0}^{\ell-1} 2^j r_j, \quad r_j \in \{0,1\}, \quad 0 \le r < p.$$

Open $c = x - y + r \pmod p$. Write the public canonical bits of c as c_j and form locally

$$[m_j]_4 = (1 - c_j) + (2c_j - 1)\,[r_j]_4\,. \tag{11}$$

Compute

$$[e]_4 = \prod_{j=0}^{\ell-1} [m_j]_4 \tag{12}$$

using a balanced tree of secure multiplications. Each factor is one precisely when the corresponding bits match. Therefore $e = 1$ iff $c = r$, equivalently $x = y$. The opened c is uniform because r is fresh and uniform. This masked equality principle appears in Nishide–Ohta, Section 6.4 [**?**]; the balanced-tree implementation and counts used here are a simple specialization.

Input representation	Factors	Secure products	Product layers
One element of $\mathbb{F}_{2^{61}-1}$	61	60	6
One element of a 128-bit prime field	128	127	7
Five independent 61-bit field limbs	305	304	9

Table 2: Derived online counts after opening the masked differences; mask production and backend verification are additional. The last row flattens the five product trees into one tree.

For 1,024 equalities in the 61-bit field, all first openings can be batched and the 61,440 secure products occupy six multiplication layers. A layer is not necessarily one network round trip; the backend may require additional phases. Fresh masks are consumed by every equality, even when inputs repeat.

9.2 Canonical masks and their real preprocessing cost

Uniform independent bits do not automatically encode a uniform canonical field element. A conceptually simple method samples ℓ shared random bits, computes a secret acceptance predicate $R < p$, opens that predicate, and repeats on rejection. Accepted masks are uniform on $\mathbb{F}_p$. This work is offline but not free. For $p = 2^{61} - 1$, only the all-ones candidate is invalid, with probability 2^{-61}; its bits must not be retained while treating its field encoding as zero. That mismatch would invalidate the equality test.

Random bits can themselves be generated by standard arithmetic preprocessing. For example, sample a shared random u, securely compute and open u^2, reject zero, choose a public square root q, and use $(1 + u/q)/2$ as a random bit over an odd prime field. This is only an algebraic description: malicious generation, consistency, and opening checks remain necessary.

Reusing an additive mask is unsafe: opening $d_1 + r$ and $d_2 + r$ reveals $d_1 - d_2$. Reusing a mask across recursive levels or duplicate groups is not justified by batch processing. Mask-bit storage and production should be measured alongside ordinary multiplication preprocessing.

9.3 A two-opening power-tuple variant

Status: Derived optimization; not claimed to be a default library protocol or a new result. The masked bit matches in (**??**) allow equality to be expressed as a low-domain polynomial. Define the mismatch count

$$H = \sum_{j=0}^{\ell-1} (1 - m_j) \in \{0, \ldots, \ell\}.$$

For $p > \ell$, let

$$P(T) = \prod_{j=1}^{\ell} \left(1 - \frac{T}{j}\right). \tag{13}$$

Then $P(0) = 1$ and $P(h) = 0$ for all integers $1 \le h \le \ell$, so $P(H) = e$.

In addition to the first mask, preprocess an independent uniform $s \in \mathbb{F}_p$ and *separate, checked degree-4 sharings*

$$[s]_4, \left[s^2\right]_4, \ldots, \left[s^\ell\right]_4.$$

Open $u = H + s$. Since u is now public, calculate the public coefficients $\beta_j(u)$ of

$$P(u - X) = \sum_{j=0}^{\ell} \beta_j(u) X^j.$$

Return

$$[e]_4 = \sum_{j=0}^{\ell} \beta_j(u) \left[s^j\right]_4 = [P(u - s)]_4 = [P(H)]_4, \qquad \left[s^0\right]_4 = 1. \tag{14}$$

The final operation is linear in already prepared degree-4 sharings. It does not raise a local Shamir share to the jth power. The second opening is uniform, even conditioned on the first transcript, because s is fresh and independent.

This gives two dependent online opening phases and no online secret products *inside the equality computation*, after correlations are ready. It shifts $O(\ell)$ products and correlated storage offline; simple sequential power generation uses $\ell - 1$ products, plus verification. Naive public coefficient shifting takes $O(\ell^2)$ local operations per equality. The first canonical-bit mask is still needed, and both types of correlations are single-use.

The tradeoff is lower online depth against preprocessing, memory, and public arithmetic. A continuously running service must replenish the power tuples, so the improvement in end-to-end throughput is unresolved. Related constant-round arithmetic techniques are classical [**?**]; the exact formulation above is recorded as a derivation from the discussion rather than an attributed measured implementation.

9.4 Bounded integer equality and field-size slack

Suppose $x, y \in [0, 2^k)$, so $d = x - y$ satisfies $|d| < 2^k$. A bounded-integer masking variant opens

$$c = d + 2^k + r_{\text{low}} + 2^k r_{\text{high}}, \tag{15}$$

where r_{low} has k shared random bits and r_{high} supplies statistical hiding. Choose the field and mask ranges so there is no modular wraparound. The low k bits of c equal those of r_{low} iff $d \equiv 0 \pmod{2^k}$, which under the bound means $d = 0$.

This allows a k-factor equality tree: $k - 1$ products and $\lceil \log_2 k \rceil$ layers after the opening. For a 28-bit address this is 27 products and five layers. However, statistical masking needs approximately $k + \sigma$ bits plus appropriate headroom in the field. A 61-bit prime cannot simultaneously provide 28-bit operands and 64 bits of integer-masking slack. The exact full-field protocol avoids this particular slack requirement by using all canonical field bits; the active verification soundness is a separate issue. For a concrete sufficient condition, choose independent uniform k-bit r_{low} and σ-bit r_{high} and take $p > 2^{k+\sigma} + 2^{k+1}$. Relative to a transcript simulated with $d = 0$, the per-test statistical privacy distance is below $2^{-\sigma}$. These errors compose across tests; correctness under the stated bounds remains exact.

MP-SPDZ's integer equality code contains masking-based paths associated with Catrina–de Hoogh [**?**, **?**]. Other paths, including a known-prime path, may use different bit decompositions. Therefore "28-bit equality costs 27 multiplications" is a count for the explicitly described construction, not a claim about every invocation of a high-level equality operator. Malicious inputs must satisfy the assumed range, either by representation or checking.

9.5 Full 256-bit keys

A 256-bit key must be encoded injectively. In $\mathbb{F}_{2^{61}-1}$, four 60-bit limbs and one 16-bit limb suffice. Independently mask each limb and combine all match factors. The generic full-field route uses $5 \cdot 61 = 305$ factors, 304 products, and nine product layers. The five initial openings can share one communication phase. This intentionally simple count does not optimize the shorter last limb.

Alternatively, sum the five mismatch counts and use the power-tuple construction with a polynomial of degree 305. It still has two opening phases, but substantially larger offline tuples. If both keys are permanently bit-shared, each bit match is $1 - x_j - y_j + 2x_j y_j$; direct equality uses $2k - 1$ products over k bits, with $1 + \lceil \log_2 k \rceil$ layers. If one operand is public, the per-bit products disappear. Permanent bit storage may be costly across the entire database.

Fingerprints can reduce comparison width only by introducing and accounting for collision probability, or by retaining an exact final check. No fingerprint parameters or adaptive-query security argument were selected in the discussion.

9.6 Other zero tests and pitfalls

Fermat gives the exact formula

$$e = 1 - (x - y)^{p-1}.$$

It requires secure squarings and multiplications with degree reduction, and has a longer dependency chain than a balanced match tree. It avoids canonical-bit mask preprocessing, so its full cost can still be competitive on a low-latency backend. Evaluate a good addition chain rather than assuming a naive binary exponentiation count.

Multiplying $x - y$ by a fresh nonzero secret random value and opening the product is an inexpensive *public* zero test. It reveals whether $x = y$ and is therefore generally unsuitable for internal ORAM membership. Neither this test nor Fermat exponentiation can be implemented by simply opening or exponentiating unreduced local Shamir products.

10 Preprocessing literature: what changes with honest majority

10.1 A useful practical reference: ATLAS

ATLAS [**?**] gives honest-majority arithmetic MPC, including malicious security with abort. Its leading full multiplication communication is about four field elements per party in the information-theoretic variant and about two with PRG-based savings; sublinear verification terms remain. Round compression trades these leading constants for approximately 4.5 and 2.5, respectively. The paper's synthetic benchmark uses a 61-bit Mersenne field and one AWS `c4.large` per party. Table 2 reports 10 million multiplications at depth 20 in 7.892 seconds for nine parties and 8.628 seconds for eleven parties. These imply roughly 1.27 and 1.16 million multiplications/s per complete party instance, not per core. The experimental variant is information-theoretic.

The author implementation's timed execution includes offline and online phases [**?**]. *Code provenance:* the inspected author checkout is revision `f7bf4193ca24e780103cd08aa9e8930e272147f8`, whose implementation uses four software threads. The neighboring party counts are a useful scale reference, not an exact ten-party result. Nor is the synthetic circuit a database benchmark. The small benchmark instances cannot hold the target database. Field width, security checks, circuit depth, memory pressure, and networking prevent a direct conversion to a modern VM's per-core capacity.

10.2 Recent honest-majority directions

The following are relevant after accepting honest majority. None supplies a verified end-to-end measurement of the proposed database.

Work	Relevance and qualification
Song–Ye, EUROCRYPT 2025 [**?**]	Malicious honest-majority MPC using sharing transformation and packed Beaver correlations. Preparation costs $O(n)$ field elements plus $O(n)$ random OLE correlations per packed triple. OLE generation is part of the cost, not a free primitive. Relevant to reducing total communication beyond plain Shamir.
Li–Xing–Yao–Yuan, PKC 2026 [**?**]	Sumcheck-based malicious verification. For Shamir, retains $O(M)$ extra computation and $O(\log M)$ extra communication, replaces certain double sharings with random sharings, and improves a soundness dependence from $O(M/\lvert\mathbb{F}\rvert)$ to $O(\log M/\lvert\mathbb{F}\rvert)$. Applicable to the ATLAS line; not a measured ten-party replacement in this discussion.

Work	Relevance and qualification
Helix, 2025 preprint [**?**]	General-n malicious honest-majority inference with compressed checks exploiting operand reuse and a revised prefix-OR primitive. Reusing a control bit across record words suggests a possible ORAM connection. Published neural-network speedups are not generic ORAM or scalar multiplication rates.
Bienstock–Escudero–Polychroniadou, CRYPTO 2026 [**?**]	CRT-based sharing gives linear total computation and communication for active corruptions with $t < (1/2 - \varepsilon)n$. Asymptotic constants and the required gap from one half must be evaluated for ten parties.
Du–Song–Ye, ASIACRYPT 2026 preprint [**?**]	Honest-majority guaranteed output delivery from pairwise random OLEs, using detectable packed sharing. Linear online communication in the OLE-hybrid model; information-theoretic preprocessing, random-oracle, and PCG instantiations have different total costs. GOD is stronger than the current working abort model.
Song–Ye, 2025 preprint revised 2026 [**?**]	Perfect security with $t < n/3$, using packed bivariate triples. At ten parties this supports at most three corruptions. Linear online communication is conditioned on preprocessing; preparing these correlations has an $\Omega(n)$ per-secret communication lower bound in the stated setting.

The field size alone does not specify malicious soundness. For instance, a check with error proportional to $M/|\mathbb{F}|$ behaves differently from one proportional to $\log M/|\mathbb{F}|$. At $M \approx 10^7$ and a 61-bit field, the former scaling is roughly 2^{-38} before constants or repetitions. This is an illustration of the need to inspect the actual theorem, not an audited failure probability for a particular implementation. Extension-field checking or repetition can alter the cost. Statistical hiding for bounded comparisons is another independent parameter.

10.3 How to read multiparty triple-generation claims

The original question asked specifically about more than two parties. Four distinctions are indispensable:

1. Raw Beaver triples do not include the authentication needed by a conventional full-threshold SPDZ backend.
2. A multiparty raw-correlation construction may have only a two-party authenticated extension or benchmark.
3. A PCG changes seed communication and local expansion; authenticated setup, checks, memory, and expansion CPU can remain substantial.
4. A protocol that avoids ordinary authenticated triples by changing online verification is not a drop-in faster triple generator for an unchanged backend.

Relevant dishonest-majority results are retained in Appendix **??** for continuity. Their relevance is secondary once a suitable honest-majority backend is allowed. Improvements for matrix triples or 1024-bit scalar fields should not be applied to 61-/128-bit scalar ORAM gates without evidence.

11 AWS cost model for 10,000 reads per second

11.1 What is known and what is missing

Known requirement: $R = 10^4$ logical reads/s, ten parties, service running continuously, with multiple workers allowed. **Missing measurement:** the complete work and traffic per lookup for a chosen construction. Therefore the honest-majority discussion supports a parameterized estimate, not a final annual bill.

The earlier dishonest-majority estimate of roughly \$0.3–1 billion/year across all parties assumed 10,000–30,000 authenticated scalar multiplications per read and extrapolated conventional LowGear-style preprocessing. Both the per-read gate budget and its ten-party throughput extrapolation were provisional. **Do not reuse that estimate as the honest-majority budget.** The workload has not been shown to require $G = 10{,}000$–$30{,}000$; those numbers are sensitivity scenarios only.

11.2 Throughput and leading multiplication traffic

Let c be average transmitted field words per party per full multiplication, w the encoded field-word size in bytes, and G the complete multiplication count per logical lookup. For an ATLAS-inspired sensitivity model choose $c = 4$:

$$M_{\text{sec}} = RG, \tag{16}$$

$$B_{\text{party/sec}} \simeq RGcw, \tag{17}$$

$$B_{\text{all/year}} \simeq nRGcw \cdot 31{,}536{,}000. \tag{18}$$

The coefficient is a leading communication proxy, not a statement that every Shamir implementation has that cost. Rounding effects, verification, openings, bit masks, permutation protocols, PRFs, framing, and other work may add traffic. If all those primitives are charged through an equivalent G, avoid counting their products again separately.

G/read	Multiplications/s	Word bytes	GB/s/party	Gbit/s/party
10,000	10^8	8	3.2	25.6
10,000	10^8	16	6.4	51.2
30,000	$3 \cdot 10^8$	8	9.6	76.8
30,000	$3 \cdot 10^8$	16	19.2	153.6

Table 4: Derived leading traffic at $c = 4$, decimal GB and Gbit. These are conditional scenarios, not measured lookup costs.

Using $c = 2$ halves this leading term but does not establish a matching compute speedup or a ready-made implementation. Similarly, arithmetic over 16-byte words cannot be assigned the measured 61-bit multiplication rate merely by doubling network bytes.

For perspective only, dividing 100–300 million multiplications/s by a nearby published rate of 1.2 million/s yields about 84–250 parallel MPC worker groups. Each group has a participant at every party. These are neither 84–250 CPU cores nor a validated deployment plan. If every group duplicates full ORAM state, per-party memory can grow into terabytes. Sharing state between workers requires a correct concurrent ORAM schedule; simply starting more processes is not free parallelism.

11.3 Compute-price snapshot

The following Linux On-Demand, shared-tenancy prices for US East (N. Virginia) were retrieved in the discussion from AWS's official offer feed [**?**]. The stored feed metadata gives publication `2026-10-09T20:31:34Z`, version `20261009203134`, with the selected rates effective 1 October 2026. They are a dated reproducibility snapshot.

Instance	vCPU	RAM	Network	USD/hour	Ten nodes/year
`c7i.8xlarge`	32	64 GiB	12.5 Gbit/s	1.42800	125,093
`r7i.16xlarge`	64	512 GiB	25 Gbit/s	4.23360	370,863
`r6in.16xlarge`	64	512 GiB	100 Gbit/s	5.57856	488,682

Table 5: Ten nodes means one node per party, 8,760 hours/year. This is an instance-price calculation, not a claim that ten nodes meet the throughput target.

For K homogeneous nodes per party at price p_{hr},

$$C_{\mathrm{compute/year}} = 10Kp_{\mathrm{hr}} \cdot 8760. \tag{19}$$

Choose K to satisfy measured computation, NIC, memory capacity, and memory bandwidth requirements simultaneously. Peak advertised network performance is not a guaranteed sustained MPC rate, and vCPU count is not physical-core count. Storage, initialization, backups, engineering, spare capacity, and operational overhead are not priced in this table.

11.4 Network placement can dominate the annual bill

AWS states that VPC peering traffic within the same physical availability zone can be free of peering data-transfer charges, including across accounts [**?**]. Availability-zone names can differ between accounts; placement must refer to the same physical zone. Cross-AZ same-region transfer is charged at \$0.01/GB for each endpoint under the cited VPC pricing, giving \$0.02 per GB of wire data when counting both billed endpoints [**?**].

Let f be the fraction of the aggregate transmitted bytes that cross zones. With decimal GB, the corresponding sensitivity model is

$$C_{\mathrm{transfer/year}} = \frac{nRGcw \cdot 31{,}536{,}000}{10^9} \cdot f \cdot 0.02. \tag{20}$$

Here aggregate transmitted bytes count each network send once; the price factor accounts for billing at both ends. Do not double-count receivers again. The displayed dollar amounts assume 10^9 bytes per pricing unit; confirm that conversion against the applicable AWS usage record before producing a budget. If the billed unit is 2^{30} bytes, the corresponding estimates are 6.87% lower. This unit assumption is separate from the much larger uncertainty in G.

Same-AZ placement removes this particular fee under the cited connectivity conditions; it does not remove NIC limits, compute costs, or shared failure domains. Different-region pricing and latency require a separate calculation. Network topology is therefore a material architectural input, not a small adjustment to a CPU-only estimate.

G/read	8-byte words, all cross-AZ	16-byte words, all cross-AZ
10,000	\$20,183,040/year	\$40,366,080/year
30,000	\$60,549,120/year	\$121,098,240/year

Table 6: All ten parties, $c = 4$, $R = 10^4$/s, $f = 1$, transfer only. Multiply by f for the modeled crossing fraction.

12 Recommended experiments and unresolved decisions

12.1 Experiments in dependency order

1. **Pin the arithmetic environment.** Select malicious-with-abort or passive security explicitly, $n = 10, t = 4$ if retaining the working threshold, field representation, comparison bounds, statistical security, and source revision. Run preprocessing and online execution together for steady-state measurements.
2. **Measure primitive costs.** Benchmark full-field equality, bounded address equality, ordering, prefix operations, public/secret selection, private permutation, and stable merging at actual metadata and payload widths. Include random-bit masks and, if used, power-tuple generation and coefficient evaluation.
3. **Implement the fair direct baseline.** Use the same dense-array layout and batch size as the ORAM candidates. Record permutation preparation, payload application, local RAM traffic, duplicate behavior, and output routing.
4. **Prototype one square-root level.** Sweep shelter size around the coefficient-aware optimum. Measure shelter lookup/maintenance and a complete rebuild separately, then verify their amortized sum over whole epochs.
5. **Add recursion selectively.** Sweep fanouts and base cutoffs. Include packed subblock extraction, duplicate position-map blocks, and forced descendant rebuilds. Compare each added level with its saved flat-base cost.
6. **Compare a tree candidate.** Start from Circuit ORAM with the published parallel eviction formulation. Separate narrow control work from wide payload movement and test safe batch scheduling. Include stash and bucket parameters in both the security and memory accounts.
7. **Measure the complete map lookup.** Add key-to-index processing, all candidate locations, full-key equality, missing-key behavior, and overflow handling. Only now determine G, bytes/read, and a realistic throughput range.
8. **Scale workers and price the service.** Determine state per worker, whether replicas or coordinated shared state are used, batch formation delay, throughput saturation, rebuild tail latency, and network placement. Replace the illustrative values in Section **??** with measured values.

12.2 Security and construction questions still open

The square-root design needs a complete argument for duplicate requests, cover probes, repeated accesses across batches, dummy exhaustion, and recursive rebuild coupling. The sorted-shelter optimization also needs a concrete secure merge with favorable costs at a highly asymmetric list ratio. A read-only application simplifies record consistency but does not eliminate these issues.

For tree ORAM, establish whether the desired batch schedule preserves the published stash bound and eviction behavior. Independent path processing can conflict in shared buckets; delayed eviction can change overflow probabilities. Word-depth improvements must be translated into the selected

arithmetic circuit.

For preprocessing, choose a malicious backend rather than combining an online rate from one protocol, a triple rate from another, and a verification argument from a third. If adopting newer packed, PCG, or sumcheck machinery, check its field, corruption threshold, setup assumptions, composability, and output-release semantics against the database's public memory-access transcript.

12.3 Decision log for the next session

Decision or question	Current state
Database scale and payload	Fixed user requirements: 2^{28} pairs, 32-byte keys and values.
Read throughput	Target 10,000/s across ten parties; parallel instances allowed.
Application writes	Excluded from the present performance target.
Honest majority	Accepted by user.
Active versus passive	Active with abort used as the working interpretation; not finalized.
Threshold	Degree-4 Shamir suggests $t = 4$ among ten; retain unless changed explicitly.
Batch size	1,024 logical requests as the main example; tune against latency and load.
Key map	Cuckoo-style dense conversion versus integrated search tree remains open.
ORAM family	Enlarged square-root and parallelized tree candidates remain open.
Base cutoff and fanout	Tune with complete batch costs, not asymptotics alone.
Equality	Explicit exact protocols available; fastest full-cost choice unmeasured.
Preprocessing backend	Honest-majority Shamir baselines first; newer methods worth evaluating.
Annual total cost	Unresolved until complete primitive and lookup measurements exist.

13 Notes for future readers and reproducibility

This is a consolidation of a research conversation, with primary references checked where needed. It preserves proposed constructions as proposals and conditional arithmetic as conditional arithmetic. It does not claim an original publication, a finished security proof, a deployed system, or an AWS quote. Prices, software behavior, and preprint versions may change.

The source is a single self-contained LaTeX file with an embedded bibliography. Compile with `latexmk -pdf batched_mpc_oram_report.tex`, or run `pdflatex` twice. No external figures, bibliography database, or local research checkout is required. Primary URLs in the bibliography make later verification possible without the chat transcript.

To continue efficiently, read Sections **??**, **??**, and **??** first. The highest-value next artifact is a reproducible benchmark of batched gather, shelter merge, rebuild, and equality on the chosen ten-party backend. That benchmark should settle the coefficient ratios in Equation (**??**) before committing to a complete ORAM construction.

A Dishonest-majority multiparty preprocessing references

This appendix retains the earlier literature question without treating it as the main deployment assumption.

Abram–Scholl, PKC 2022 [?]. Ring-LPN-based multiparty authenticated triples for SPDZ over large prime fields, including a malicious distributed setup. It is a genuine multiparty reference; party-count-dependent setup and seed costs require attention. No implementation from this work was used in the discussion.

Multipars, PoPETs 2024 [?]. Authenticated multiparty preprocessing over $\mathbb{Z}_{2^k}$ using linearly homomorphic encryption, with public code. The concrete evaluation discussed was two-party; the protocol's multiparty applicability does not supply a ten-party throughput measurement. Rings also change the equality/encoding choices.

MatriGear, IEEE S&P 2025 [?]. Improves authenticated matrix and large-prime-field triple generation through HE packing. Its reported 3.6-fold scalar improvement concerns a 1024-bit prime; the 34-fold result concerns 128×128 matrix triples. Neither is a speedup factor for 128-bit scalar ORAM multiplications.

Boyle et al., CRYPTO 2026 [?]. *Dishonest-Majority Secure Computation via PIR-Authenticated Multiplication Triples* bypasses ordinary AMT generation through a different authentication strategy built around raw PCG correlations and authenticated PIR. Multiparty extensions are relevant, but this is not an unchanged SPDZ backend with a faster drop-in AMT factory. No ten-party database benchmark was established.

Li–Song, ASIACRYPT 2026 preprint [?]. *Malicious Security Comes Free in SPDZ* achieves a leading $4n$ words per multiplication in the maximum-corruption setting using tensor-product PCG correlations and recursive checking. Additional local computation remains. Its verification architecture differs from authenticating every wire in conventional SPDZ.

Recent PCG papers on arbitrary finite fields, $\mathbb{Z}/p^k\mathbb{Z}$, sparse LPN, and Boolean triples were also examined [?, ?, ?, ?]. Their multiparty raw-correlation results must be distinguished from authenticated variants that may be only two-party. Similarly, two-party batch-authenticated PCG speedups [?] are not evidence of multiparty authenticated throughput. Parameter revisions following attacks on assumptions must be respected; old slide throughput should not override a revised paper's parameters.

B Small reproducible numerical calculations

The following Python reproduces the report's conditional arithmetic; it is not an MPC benchmark. Network units are decimal, while storage GiB/MiB is binary.

```
from math import sqrt, log1p, expm1
N, Q, rate, parties = 2**28, 1024, 10_000, 10
seconds = 365 * 24 * 3600
for D, b in [(N, Q), (2*N, 2*Q)]:
    shelter = sqrt(D*b)
    print("S, batches/epoch, seconds/epoch:",
          shelter, shelter/b, shelter/b*Q/rate)
    print("ideal optimized/direct ratio:", 2*sqrt(b/D))
for gates in [10_000, 30_000]:
    for word_bytes in [8, 16]:
        sent = rate * gates * 4 * word_bytes # per party/sec
        cross_az_cost = sent * parties * seconds / 1e9 * 0.02
```

```
        print(gates, word_bytes, sent/1e9, sent*8/1e9,
              cross_az_cost)
for name, hourly in [("c7i.8xlarge", 1.428),
                     ("r7i.16xlarge", 4.2336),
                     ("r6in.16xlarge", 5.57856)]:
    print(name, hourly * 8760 * parties)
# Expected union of 1024 paths, depths 0 through 28 inclusive:
nodes = 1 + sum(2**d * -expm1(Q*log1p(-2**(-d)))
                for d in range(1, 29))
print("expected distinct buckets:", nodes)
```

C Proposed scan circuits for Circuit ORAM eviction

Status: Derived circuit reformulations; not a complete batched ORAM proof. This appendix retains concrete ideas from the discussion. The published Chan–Chung–Shi construction remains the established reference. The scans below require implementation-level checking against the chosen Circuit ORAM semantics, including stash handling, tie-breaking, and bucket-local selection. They do not establish a stash bound for delayed batch eviction.

C.1 Eligibility from a public path

For public eviction-path bits $p_1, \ldots, p_h$ and shared leaf bits $x_1, \ldots, x_h$, set $m_j = x_j$ if $p_j = 1$ and $m_j = 1 - x_j$ otherwise. Then

$$E_d = \prod_{j=1}^{d} m_j, \qquad g = \sum_{d=1}^{h} E_d$$

give the allowed-depth indicators and common-prefix length. A work-efficient prefix-product scan has $O(h)$ multiplication work and $O(\log h)$ depth. Authenticated bit availability is an assumption; conversion from compact field labels must be charged if needed.

C.2 A proposed target planner

Number path locations $0, \ldots, L-1$, including a stash location only with a consistent renumbering of all permissible destinations. Let g_i be the maximum permissible destination among location i's real blocks, or -1 if none, and let e_i indicate an originally empty slot. Ties favor the shallower source. Define

$$\begin{aligned} G_i &= \max_{0 \le j \le i} g_j, \qquad G_{-1} = -1, \\ r_i &= [g_i > G_{i-1}] \wedge [g_i \ge 0], \\ c_d &= [G_{d-1} \ge d] \quad (d \ge 1). \end{aligned}$$

Here brackets denote secret predicates, not public revelations. A source can win the greedy best-ancestor choice only if it sets a new prefix record.

Compute backward from $u_{L-1} = 0$:

$$u_i = c_{i+1} \wedge (e_{i+1} \vee u_{i+1}), \qquad a_i = r_i \wedge u_i, \qquad v_i = e_i \vee a_i.$$

The intended meaning is that a_i selects exporting sources and v_i indicates a vacancy after their export. For the target scan use a distinguished absent value $\perp$, represented by a separate presence bit. Put $t_{L-1} = \perp$ and, with $d = i + 1$, compute

$$t_i = \begin{cases} \perp, & \neg c_d, \\ d, & c_d \wedge r_d \wedge v_d, \\ \perp, & c_d \wedge r_d \wedge \neg v_d, \\ t_{i+1}, & c_d \wedge \neg r_d \wedge (t_{i+1} \neq \perp), \\ d, & c_d \wedge \neg r_d \wedge (t_{i+1} = \perp) \wedge v_d, \\ \perp, & \text{otherwise.} \end{cases}$$

Finally $\text{target}_i = t_i$ for exporters and $\perp$ otherwise.

Reasoning to retain. Let the record sources be $q_0 < q_1 < \cdots < q_m$, with $q_{m+1} = L$. Source q_j is the best eligible ancestor on the interval

$$I_j = (q_j, \min\{g_{q_j}, q_{j+1}, L - 1\}].$$

A vacancy comes from an original empty slot in this interval, or from the next record source at its endpoint if that source itself exports. Coverage gaps prevent propagation. The u recurrence captures availability, and the t recurrence chooses the deepest eligible vacancy, resetting at a gap or the next record source. This is the intended correspondence to greedy sequential eviction; it should be formalized against the exact bucket/stash invariants before use.

C.3 Associative summaries for the backward passes

The u step is an affine Boolean map over the arithmetic field:

$$f_d(x) = A_d + B_d x, \qquad A_d = c_d e_d, \quad B_d = c_d - A_d.$$

Composition is associative:

$$(A, B) \circ (A', B') = (A + BA', BB').$$

For the target step, use a reset bit R and optional candidate C:

$$F_{R,C}(x) = \begin{cases} C, & R = 1, \\ x, & R = 0 \text{ and } x \neq \perp, \\ C, & R = 0 \text{ and } x = \perp. \end{cases}$$

At destination d, take $R_d = \neg c_d \vee r_d$ and let $C_d = d$ if $c_d \wedge v_d$, otherwise $\perp$. Composition yields

$$F_{R,C} \circ F_{S,D} = F_{R \vee S,\ C'}, \qquad C' = \begin{cases} C, & R = 1, \\ D, & R = 0 \text{ and } D \neq \perp, \\ C, & \text{otherwise.} \end{cases}$$

These summaries support $O(L)$ scan-combination work and $O(\log L)$ depth. Prefix maxima, small comparisons, and bucket candidate selection remain additional work. The scan's constant depth is not one multiplication layer per doubling step for the entire eviction.

C.4 Parallel payload propagation

Once targets are known, preselect each bucket's original export block D_i and form optional records $(a_i, D_i, \text{target}_i)$. An exclusive prefix scan selecting the rightmost valid record gives the most recent exporting source strictly above each bucket. Accept its payload at bucket i precisely when its target is i.

The intended invariant is that the greedy single-carried-block eviction has nonoverlapping source-to-destination intervals. An earlier block already deposited has a smaller target and is rejected by the target equality. This avoids an L-by-L payload routing matrix. With bucket capacity Z and payload width w field words, the proposed work is $O(LZw + Lw)$ and payload scan depth $O(\log L)$, plus control and local selection. This is a circuit-level proposal, not a claim of measured improvement. Compare its constants with Titanium-style sequential swaps rather than multiplying two independent advertised speedups.

References

[1] Peeter Laud. *Parallel Oblivious Array Access for Secure Multiparty Computation and Privacy-Preserving Minimum Spanning Trees*. Proceedings on Privacy Enhancing Technologies, 2015(2):188–205, 2015. https://petsymposium.org/popets/2015/popets-2015-0011.pdf.

[2] Brett Hemenway Falk and Rafail Ostrovsky. *Secure Merge with $O(n \log\log n)$ Secure Operations*. ITC 2021, LIPIcs 199, article 7. https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITC.2021.7.

[3] Mark Blunk, Paul Bunn, Samuel Dittmer, Steve Lu, and Rafail Ostrovsky. *Linear-Time Secure Merge in $O(\log\log n)$ Rounds*. ITC 2025, article 7. https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITC.2025.7.

[4] Suvradip Chakraborty, Stanislav Peceny, Srinivasan Raghuraman, and Peter Rindal. *Logstar: Efficient Linear* Time Secure Merge*. Cryptology ePrint Archive, Paper 2024/159, revised September 2026. https://eprint.iacr.org/2024/159.

[5] Jiacheng Gao, Yuan Zhang, Sheng Zhong, and Changyu Dong. *Free Linear Online Phase for Secure Multiparty Shuffle*. Cryptology ePrint Archive, Paper 2024/1936, revised September 2026; ASIACRYPT 2026 version. https://eprint.iacr.org/2024/1936.

[6] Samee Zahur, Xiao Wang, Mariana Raykova, Adrià Gascón, Jack Doerner, David Evans, and Jonathan Katz. *Revisiting Square-Root ORAM: Efficient Random Access in Multi-Party Computation*. IEEE Symposium on Security and Privacy, 2016. https://www.cs.umd.edu/~jkatz/papers/sqoram.pdf.

[7] Xiao Wang, T-H. Hubert Chan, and Elaine Shi. *Circuit ORAM: On Tightness of the Goldreich-Ostrovsky Lower Bound*. ACM CCS 2015, pp. 850–861; full version, Cryptology ePrint Archive, Paper 2014/672. https://eprint.iacr.org/2014/672.

[8] T-H. Hubert Chan, Kai-Min Chung, and Elaine Shi. *On the Depth of Oblivious Parallel RAM*. ASIACRYPT 2017; full version, Cryptology ePrint Archive, Paper 2017/861. https://eprint.iacr.org/2017/861.

[9] Weikeng Chen, Thang Hoang, Jorge Guajardo, and Attila A. Yavuz. *Titanium: A Metadata-Hiding File-Sharing System with Malicious Security*. NDSS 2022; full version, Cryptology ePrint Archive, Paper 2022/051. https://eprint.iacr.org/2022/051.

[10] Ling Ren, Christopher Fletcher, Albert Kwon, Emil Stefanov, Elaine Shi, Marten van Dijk, and Srinivas Devadas. *Constants Count: Practical Improvements to Oblivious RAM*. USENIX Security, 2015. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/ren-ling.

[11] Stanislaw Jarecki and Boyang Wei. *3PC ORAM with Low Latency, Low Bandwidth, and Fast Batch Retrieval*. ACNS 2018, pp. 360–378. https://eprint.iacr.org/2018/347.

[12] Zhengyuan Su, Qi Pang, Simon Beyzerov, and Wenting Zheng. *FABLE: Batched Evaluation on Confidential Lookup Tables in 2PC*. USENIX Security, 2025. https://www.usenix.org/conference/usenixsecurity25/presentation/su-zhengyuan.

[13] Afonso Tinoco, Sixiang Gao, and Elaine Shi. *EnigMap: External-Memory Oblivious Map for Secure Enclaves*. USENIX Security, 2023, pp. 4033–4050. https://www.usenix.org/conference/usenixsecurity23/presentation/tinoco.

[14] MP-SPDZ contributors. *MP-SPDZ Documentation: Getting Started and Protocol Overview*. Maintained software documentation; accessed 13 October 2026. https://mp-spdz.readthedocs.io/en/latest/readme.html.

[15] MP-SPDZ contributors. *MP-SPDZ: Shamir Protocol Options*. Maintained source code; accessed 13 October 2026. https://github.com/data61/MP-SPDZ/blob/master/Protocols/ShamirOptions.cpp.

[16] Yehuda Lindell and Ariel Nof. *A Framework for Constructing Fast MPC over Arithmetic Circuits with Malicious Adversaries and an Honest-Majority*. ACM CCS, 2017. https://doi.org/10.1145/3133956.3133999.

[17] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. *Fast Large-Scale Honest-Majority MPC for Malicious Adversaries*. CRYPTO 2018, pp. 34–64. https://eprint.iacr.org/2018/570.

[18] Takashi Nishide and Kazuo Ohta. *Multiparty Computation for Interval, Equality, and Comparison Without Bit-Decomposition Protocol*. PKC 2007, pp. 343–360. https://doi.org/10.1007/978-3-540-71677-8_23.

[19] Ivan Damgård et al. *Unconditionally Secure Constant-Rounds Multi-party Computation for Equality, Comparison, Bits and Exponentiation*. TCC 2006, pp. 285–304. https://doi.org/10.1007/11681878_15.

[20] Octavian Catrina and Sebastiaan de Hoogh. *Improved Primitives for Secure Multiparty Integer Computation*. SCN 2010, pp. 182–199. https://doi.org/10.1007/978-3-642-15317-4_13.

[21] MP-SPDZ contributors. *MP-SPDZ Compiler: Equality-to-Zero and Nonlinear Arithmetic Implementations*. Maintained source code, `Compiler/floatingpoint.py` and `Compiler/non_linear.py`; accessed 13 October 2026. https://github.com/data61/MP-SPDZ/blob/master/Compiler/floatingpoint.py; https://github.com/data61/MP-SPDZ/blob/master/Compiler/non_linear.py.

[22] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. *ATLAS: Efficient and Scalable MPC in the Honest Majority Setting*. CRYPTO 2021; full version, Cryptology ePrint Archive, Paper 2021/833. https://eprint.iacr.org/2021/833.

[23] ATLAS authors. *ATLAS: Reference Implementation and README*. Maintained software repository; accessed 13 October 2026. https://github.com/hanjunli/ATLAS.

[24] Yifan Song and Xiaxi Ye. *Honest Majority MPC with $\tilde{O}(|C|)$ Communication in Minicrypt*. EUROCRYPT 2025; Cryptology ePrint Archive, Paper 2025/283. https://eprint.iacr.org/2025/283.

[25] Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. *On the Power of Sumcheck in Secure Multiparty Computation*. PKC 2026; full version, Cryptology ePrint Archive, Paper 2025/177. https://eprint.iacr.org/2025/177.

[26] Yansong Zhang, Xiaojun Chen, Qinghui Zhang, Xudong Chen, and Ye Dong. *Helix: Scalable Multi-Party Machine Learning Inference against Malicious Adversaries*. Preprint, Cryptology ePrint Archive, Paper 2025/347, 2025. https://eprint.iacr.org/2025/347.

[27] Alexander Bienstock, Daniel Escudero, and Antigoni Polychroniadou. *Actively Secure MPC with $O(|C|)$ Computation and Communication via CRT*. CRYPTO 2026; Cryptology ePrint Archive, Paper 2026/1270. https://eprint.iacr.org/2026/1270.

[28] Zonghang Du, Yifan Song, and Xiaxi Ye. *Honest Majority MPC Achieving GOD with $O(|C|)$ Communication From Pairwise Random OLE Correlations*. ASIACRYPT 2026 preprint, Cryptology ePrint Archive, Paper 2026/1948. https://eprint.iacr.org/2026/1948.

[29] Yifan Song and Xiaxi Ye. *Scalable Secure Multiparty Computation with Perfect Security from Preprocessing*. Preprint, Cryptology ePrint Archive, Paper 2025/1406, revised 2026. https://eprint.iacr.org/2025/1406.

[30] Amazon Web Services. *Amazon EC2 On-Demand Price List: US East (N. Virginia)*. Maintained price feed; publication snapshot 9 October 2026, 20:31:34 UTC, version 20261009203134; accessed 13 October 2026. https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/us-east-1/index.csv.

[31] Amazon Web Services. *What Is VPC Peering?* Amazon Virtual Private Cloud User Guide, including pricing for a VPC peering connection. Maintained documentation; accessed 13 October 2026. https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html.

[32] Amazon Web Services. *Amazon VPC Pricing*. Maintained pricing documentation; accessed 13 October 2026. https://aws.amazon.com/vpc/pricing/.

[33] Damiano Abram and Peter Scholl. *Low-Communication Multiparty Triple Generation for SPDZ from Ring-LPN*. PKC 2022, pp. 221–251. https://iacr.org/archive/pkc2022/131770119/131770119.pdf.

[34] Sebastian Hasler, Pascal Reisert, Marc Rivinius, and Ralf Küsters. *Multipars: Reduced-Communication MPC over $\mathbb{Z}_{2^k}$*. Proceedings on Privacy Enhancing Technologies, 2024(2):5–28, 2024. https://petsymposium.org/popets/2024/popets-2024-0038.php.

[35] HyunHo Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. *MatriGear: Accelerating Authenticated Matrix Triple Generation with Scalable Prime Fields via Optimized HE Packing*. IEEE Symposium on Security and Privacy, 2025, pp. 2453–2471. https://doi.org/10.1109/SP61157.2025.00174.

[36] Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Ariel Nof. *Dishonest-Majority Secure Computation via PIR-Authenticated Multiplication Triples*. CRYPTO 2026; Cryptology ePrint Archive, Paper 2026/257. https://eprint.iacr.org/2026/257.

[37] Junru Li and Yifan Song. *Malicious Security Comes Free in SPDZ*. ASIACRYPT 2026 preprint, Cryptology ePrint Archive, Paper 2026/283. https://eprint.iacr.org/2026/283.

[38] Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. *Efficient Pseudorandom Correlation Generators for Any Finite Field*. EUROCRYPT 2025; Cryptology ePrint Archive, Paper 2025/169. https://eprint.iacr.org/2025/169.

[39] Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. *Efficient Pseudorandom Correlation Generators over $\mathbb{Z}/p^k\mathbb{Z}$*. CRYPTO 2025; Cryptology ePrint Archive, Paper 2025/1223. https://eprint.iacr.org/2025/1223.

[40] Sebastian Hasler and Pascal Reisert. *Pseudorandom Correlation Functions for Multiparty Beaver Triples from Sparse LPN*. Preprint, Cryptology ePrint Archive, Paper 2025/2002, 2025. https://eprint.iacr.org/2025/2002.

[41] Peihan Miao, Alice Murphy, Akshayaram Srinivasan, and Max Tromanhauser. *Pseudorandom Correlation Generators for Multiparty Beaver Triples over* $\mathbb{F}_2$. Cryptology ePrint Archive, Paper 2025/1182, 2025. https://eprint.iacr.org/2025/1182.

[42] Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Ariel Nof. *Fast PCGs for Batch-Authenticated Multiplication Triples*. CRYPTO 2026, pp. 495–526. https://link.springer.com/chapter/10.1007/978-3-032-35418-1_16.

[43] Wenhao Zhang, Xiao Wang, and Chenkai Weng. *Efficient Mixed-Mode Oblivious RAMs*. Cryptology ePrint Archive, Paper 2025/1057, 2025. https://eprint.iacr.org/2025/1057.

[44] Xiao Shaun Wang et al. *Oblivious Data Structures*. ACM CCS 2014; full version, Cryptology ePrint Archive, Paper 2014/185. https://eprint.iacr.org/2014/185.

[45] Rasmus Pagh and Flemming Friche Rodler. *Cuckoo Hashing*. Journal of Algorithms, 51(2):122–144, 2004. https://www.rasmuspagh.net/papers/cuckoo-jour.pdf.

[46] Martin Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. *MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity*. ASIACRYPT 2016; Cryptology ePrint Archive, Paper 2016/492. https://eprint.iacr.org/2016/492.

[47] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. USENIX Security, 2021. https://www.usenix.org/conference/usenixsecurity21/presentation/grassi.

[48] Gilad Asharov et al. *Efficient Secure Three-Party Sorting with Applications to Data Analysis and Heavy Hitters*. ACM CCS 2022, pp. 125–138. https://eprint.iacr.org/2022/1595.

[49] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, and Junichi Tomida. *Secure Statistical Analysis on Multiple Datasets: Join and Group-By*. Cryptology ePrint Archive, Paper 2024/141, 2024; major revision of the ACM CCS 2023 paper, with a revised author list. https://eprint.iacr.org/2024/141.

[50] Gilad Asharov, Ilan Komargodski, Wei-Kai Lin, Enoch Peserico, and Elaine Shi. *Oblivious Parallel Tight Compaction*. ITC 2020; Cryptology ePrint Archive, Paper 2020/125. https://eprint.iacr.org/2020/125.